

COMP2054-ADE:

ADE Lec05

Remaining comments on Big Oh family

Rules for little-oh proofs and other advanced usages

Lecturer: Andrew Parkes

[andrew.parkes@Nottingham.ac.uk](mailto:andrew.parkes@Nottingham.ac.uk)

from <http://www.cs.nott.ac.uk/~pszajp/>

# Motivation:

Later in the module we will meet the “Master theorem”. Here is a summary from a previous C/W

**Master theorem:** Given  $T(n) = a T(n/b) + f(n)$  and  $T(1)=1$ .

Case 1: If  $f(n)$  is  $O(n^c)$  for some  $c$ , with  $c < \log_b(a)$   
then  $T(n)$  is  $\Theta(n^{\log_b(a)})$

Case 2: If  $f(n)$  is  $\Theta(n^c (\log n)^k)$  for some  $k \geq 0$ , and with  $c = \log_b(a)$   
then  $T(n)$  is  $\Theta(n^c (\log n)^{k+1})$

Case 3: If  $f(n)$  is  $\Omega(n^c)$  for some  $c$ , with  $c > \log_b(a)$   
then  $T(n)$  is  $\Theta(f(n))$

Shows that need good knowledge of logs and the Big-Oh family

# Comments

- These slides are meant for advanced understanding (but still may be part of assessments).
  - Some parts have been stated elsewhere
  - The rules are methods for little-oh that might be useful.
    - Note: if a question asks for “from the definition” then such rules should not be used.

# “Multiplication Rule” for function\*little-Oh

- Suppose  $h(n)$  is (strictly) positive,  
and  
 $f(n)$  is  $o(g(n))$  [[little-oh]]
- Then, from the definition, we know  
For all  $c > 0$  exists  $n_0$   $f(n) < c g(n)$  for all  $n \geq n_0$
- Then multiplying by  $h(n)$  gives  
For all  $c > 0$  exists  $n_0$ .  $h(n) f(n) < c h(n) g(n)$  for all  $n \geq n_0$
- Which is precisely:  $h(n) f(n)$  is  $o(h(n) g(n))$
- E.g. can multiply “1 is  $o(n)$ ” by  $h(n)=n$  to get “ $n$  is  $o(n^2)$ ”
- (Note: for brevity, we may suppress extra conditions, such as needing  $h(n) > 0$ , as they will be obvious.)

# “Multiplication Rule” for big-Oh \* little-Oh ?

Suppose  $a, c, d$  are all strictly positive numbers

If  $c < d$

then we can multiply by  $a$  to get

$$a*c < a*d$$

Given the mnemonic “ $\leq$ ”  $\sim$  “Big-Oh” and “ $<$ ”  $\sim$  “little-oh”, it is reasonable to expect to be able to multiply

“a function with a Big-Oh, and one with a little-oh to get a little-oh”.

But is this actually correct?

# “Multiplication Rule” for big-Oh \* little-Oh

- Suppose
  - $f_1(n)$  is  $O(g_1(n))$  [[Big-Oh]]
  - $f_2(n)$  is  $o(g_2(n))$  [[little-oh]]
- Then, from the definition, there exist positive constants  $c_1 > 0$ ,  $n_1$  such that
  1. Exists  $c_1 > 0$  exists  $n_1$   $f_1(n) \leq c_1 g_1(n)$  for all  $n \geq n_1$
  2. For all  $c_2 > 0$  exists  $n_2$   $f_2(n) < c_2 g_2(n)$  for all  $n \geq n_2$
- Then multiplying gives
  - Exists  $c_1$  for all  $c_2$ . exists  $n_1$  exists  $n_2$ ,  $n_0 = \max(n_1, n_2)$ ,  
 $f_1(n) f_2(n) < c_1 c_2 g_1(n) g_2(n)$  for all  $n \geq n_0$
- But for any (strictly) positive value of  $c_1$ , then “for all  $c_2 > 0$ ” captures the same as “for all  $c_1 * c_2 > 0$ ”.
- Hence:  $f_1(n) f_2(n)$  is  $o(g_1(n) g_2(n))$
- E.g.  $n$  is  $O(n)$ , and  $1$  is  $o(\log n)$ , hence  $n$  is  $o(n \log n)$
- E.g.  $n$  is  $O(n)$ , and  $\log(n)$  is  $o(n)$ , hence  $n \log n$  is  $o(n^2)$

# Advanced Usages of Big-Oh

- **Some of the following slides on just a reminder of work covered before**
- They are included for
  - The ways of thinking about Big-oh family
  - Usages that might be seen in advanced academic literature, and in mathematical literature

# Usages as sets of functions

[https://en.wikipedia.org/wiki/Stirling%27s\\_approximation](https://en.wikipedia.org/wiki/Stirling%27s_approximation)

$$\ln(n!) = n \ln n - n + O(\ln(n))$$

(Where “ln” is the “natural log” using the base e)

This seems like a weird usage, and is a bit of an abuse of notation.

It means that the extra term, which one might think of as the “error”, is “some function that is  $O(\ln n)$ ”

$$\ln(n!) = n \ln n - n + \text{err}(n)$$

And with  $\text{err}(n) \in O(\ln(n))$

The usage is to have some function that contains a set,  $S$ , such as  $O(\dots)$ , and then the intention is that within the function it means “some element from  $S$ ”.



# Usages as sets of functions (2)

Sometimes in (advanced theory) papers might see expressions such as  $n^{O(1)}$

If “big-Oh” only means “worst case of an algorithm” then this is nonsensical.

But if the big-Oh family are used to refer to sets of functions then we can take  $n^{O(1)}$  to mean

$$\{ n^{f(n)} \mid f(n) \in O(1) \}$$

e.g. including  $f(n)=1$ ,  $f(n)=2$ ,  $f(n)=3$ , .... So  $n^{O(1)}$  includes  $n^1$ ,  $n^2$ ,  $n^3$ , ...  
So,  $n^{O(1)}$  is interpreted as

“any function that is no worse than (Big-Oh of) some power law”

(Though this is a slight abuse/extension of notation, it can be useful).

# Usages as sets of functions (3)

To capture

“any function that is no worse than (Big-Oh of) some power law”

We could also think of it as a union over all  $k \geq 1$  of the sets  $O(n^k)$

$$\bigcup_{k \geq 1} O(n^k)$$

This is rarely (if ever) used, but you should be able to interpret it.

Aside:

If the functions refer to “worst-case runtime of the best algorithm for a problem” then it essentially defines **P** = “problems that can be solved in polynomial time”, as in the famous “**P** vs. **NP**” problem.

Note:

**NP** stands for “**N**on-deterministic **P**olynomial time”, like DFAs vs. NFAs.  
**NP** does NOT stand for “Non-Polynomial”!

# Usages as sets of functions (4)

$1 - o(1)$  would then mean a set of functions:

$$\{ 1 - f(n) \mid f(n) \in o(1) \text{ (and } f(n) \text{ is positive)} \}$$

Hence

forall  $c > 0$ . exists  $n_0$ . s.t. forall  $n \geq n_0$ .  $0 \leq f(n) < c$

That is:

However small we pick  $c$ , then eventually  $f(n)$ , becomes, and stays, smaller than  $c$ .

This is the same meaning as

$f(n)$  tends to zero as  $n$  goes to infinity.

So “ $1 - o(1)$ ” is interpreted as “any function which approaches 1 (from below) in the limit of  $n$  becoming large” – e.g.  $(1 - 1/n)$

# Exponentials: effect of the base

- From the definitions, prove or disprove
  1.  $2^n$  is  $O(3^n)$
  2.  $3^n$  is  $O(2^n)$
  3.  $2^n$  is  $\Omega(3^n)$
  4.  $3^n$  is  $\Omega(2^n)$
  5.  $2^n$  is  $o(3^n)$
  6.  $3^n$  is  $o(2^n)$

# Exponentials: $2^n$ vs $3^n$

From the definitions, prove or disprove  
 $2^n$  is  $O(3^n)$

Ans: Yes. E.g. take  $c=1$   $n_0=1$   
we just need  
 $2^n \leq 1 \cdot 3^n$   
or need  $1 \leq (3/2)^n$   
which is true for  $n \geq 1$   
Proven.

# Exponentials: $3^n$ vs $2^n$

From the definitions, prove or disprove

$3^n$  is  $O(2^n)$

Ans: Disprove. We would need

$$3^n \leq c \cdot 2^n$$

i.e.  $(3/2)^n \leq c$

which will fail for any fixed  $c$ , at large enough  $n$

# Exponentials: effect of the base (2)

OFFLINE: Similarly, from the definitions, (dis)prove the Big-Omega and little-oh cases. Should be able to show:

1.  $2^n$  is  $O(3^n)$
2.  $3^n$  is NOT  $O(2^n)$  “cannot ignore the base”
3.  $2^n$  is NOT  $\Omega(3^n)$
4.  $3^n$  is  $\Omega(2^n)$
5.  $2^n$  is  $o(3^n)$  “smaller base is strictly better”
6.  $3^n$  is NOT  $o(2^n)$

# Exponentials

- If we have an exponential  $f(n) = b^n$  with  $b > 1$ , then “can we ignore the  $b$  and just think of it as  $2^n$  ?”
- Answer: No we cannot ignore the base.  
Use  $b = 2^{\lg(b)}$
- Then:  $b^n$   
$$= (2^{\lg(b)})^n = 2^{\lg(b) * n}$$
$$= (2^n)^{\lg(b)}$$
- So, changing  $b$  is like changing the power  $k$  in  $n^k$   
– and Big-Oh family does not “ignore powers”



# “Exponential in $n$ ” ?

- What might the following mean?  
 $O(2^{O(n)})$
- $2^{O(n)}$  includes  $2^n$
- It also included  $2^{(an)}$ 
  - If pick  $a > 0$  then it is  $(2^a)^n$  write  $b=2^a$  and get  $b > 1$ , and  $b^n$
  - Then the  $O(\cdot)$  wrapper allows overall constants – it might not be needed ??
- So, it just translates to “no worse than exponential in  $n$ ”.
  - Normally would just write  $2^{O(n)}$

# “Exponential in n”

To capture

“any function that is no worse than some exponential in n

We could also think of it as a union over all  $b \geq 1$  of the sets  $O(b^n)$

$$\bigcup_{b \geq 1} O(b^n)$$

Note: only need integer b. Exercise (offline): why?

NOTE: it is important that  $b=1$  is not allowed as  $1^n = 1$  and so would include  $O(1)$  which is not intended

This is rarely used, but you should be able to interpret it.  
E.g. it would include  $100^n$  and also  $1.0001^n$ ,

# Summary of Big-Oh family

- Need to be familiar with
  - Definitions, and “prove/disprove from the definition”
  - Usage of “rules”
    - Good understanding of logs and exponents
  - Everyday usage of Big-Oh family
  - Advanced usage using interpretations as “sets of functions”